

CO3-AT1-Debugging

[← Back](#)

QUESTIONS

Q1 ✓
Multiple Catch Blocks Ordering
25 marks**Q2** ✓
Basic Thread Creation and Start
25 marks**Q3** ✓
try with resources Misuse
25 marks**Q4** ✓
Producer-Consumer with
BlockingQueue
25 marks

4/4 answered

Q1 Multiple Catch Blocks Ordering

25 marks

```
public class MultiCatchDemo {  
    public static void main(String[] args) {  
        try {  
            processData(args);  
        } catch (Exception e) {  
            System.out.println("General error: " + e.getMessage());  
        } catch (ArithmeticException e) {  
            System.out.println("Math error");  
        } catch (NumberFormatException e) {  
            System.out.println("Format error");  
        }  
    }  
    try {  
        int x = 10 / 0;  
    } catch (ArithmeticException | ArithmeticException e) {  
        System.out.println(e);  
    }  
    try {  
        Object o = "text";  
        Integer i = (Integer) o;  
    } catch (ClassCastException e) {  
        System.out.println("Cast failed");  
    }  
}  
  
static void processData(String[] args) {  
    int val = Integer.parseInt(args[0]);  
    System.out.println(100 / val);  
}
```

Your Code

```
1 public class MultiCatchDemo {  
2     public static void main(String[] args) {  
3         try {  
4             processData(args);  
5         } catch (ArithmeticException e) {  
6             System.out.println("Math error");  
7         } catch (NumberFormatException e) {  
8             System.out.println("Format error");  
9         } catch (Exception e) {  
10            System.out.println("General error: " + e.getMessage());  
11        }  
12        try {  
13            int x = 10 / 0;  
14        } catch (ArithmeticException e) {  
15            System.out.println(e);  
16        }  
17        try {  
18            Object o = "text";  
19            Integer i = (Integer) o;
```

Input

stdin input

Output

```
java.lang.ArithmeticException: / by  
zero  
Cast failed
```

CO3-AT1-Debugging

[← Back](#)

QUESTIONS

Q1
Multiple Catch Blocks Ordering
25 marks**Q2**
Basic Thread Creation and Start
25 marks**Q3**
try with resources Misuse
25 marks**Q4**
Producer-Consumer with
BlockingQueue
25 marks

4/4 answered

Q2 Basic Thread Creation and Start

25 marks

```
public class ThreadBasics {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyTask());  
        t1.run();  
        MyTask task = new MyTask();  
        task.start();  
  
        Thread t2 = new Thread(task);  
        t2.start();  
        t2.start();  
  
        System.out.println(t1.getName());  
  
        try {  
            t2.join()  
        } catch (InterruptedException e) {  
        }  
    }  
}  
  
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Task running on: " + Thread.currentThread().getName());  
    }  
}
```

Your Code

```
1 public class ThreadBasics {  
2     public static void main(String[] args) {  
3         Thread t1 = new Thread(new MyTask());  
4         t1.start();  
5         MyTask task = new MyTask();  
6         Thread t2 = new Thread(task);  
7         t2.start();  
8         System.out.println(t1.getName());  
9         try {  
10            t2.join();  
11        } catch (InterruptedException e) {  
12            Thread.currentThread().interrupt();  
13        }  
14    }  
15 }  
16 class MyTask implements Runnable {  
17     public void run() {  
18         System.out.println("Task running on: " + Thread.currentThread().getName());  
19     }  
20 }  
21
```

Input

stdin input

Output

```
Thread-0  
Task running on: Thread-1  
Task running on: Thread-0
```

Run

Save

QUESTIONS

Q1
Multiple Catch Blocks Ordering
25 marks**Q2**
Basic Thread Creation and Start
25 marks**Q3**
try with resources Misuse
25 marks**Q4**
Producer-Consumer with
BlockingQueue
25 marks

4/4 answered

Q3 try-with-resources Misuse

25 marks

```
import java.io.*;

public class TryWithResourcesDemo {
    public static void main(String[] args) {
        BufferedReader reader = new BufferedReader(new FileReader("data.txt"));
        try (reader) {
            String line = reader.readLine();
            System.out.println(line);
        } catch (IOException e) {
            e.printStackTrace();
        }

        try (FileWriter writer = new FileWriter("out.txt")) {
            writer.write("Hello");
        }

        class NotCloseable {
            void write() {}
        }

        try (NotCloseable nc = new NotCloseable()) {
            nc.write();
        }

        try (FileReader fr1 = new FileReader("a.txt");
            FileReader fr2 = new FileReader("b.txt"))
        {
            fr1.close();
            fr2.close(); close (double-close, though usually harmless it's a logic bug/anti-pattern here compounded by bug 5)
        } catch (IOException e) {
        } finally {
            fr1.close();
        }
    }
}
```

Your Code

```
1 import java.io.*;
2 public class TryWithResourcesDemo {
3     public static void main(String[] args) {
4         try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
5             String line = reader.readLine();
6             System.out.println(line);
7         } catch (IOException e) {
8             e.printStackTrace();
9         }
10        try (FileWriter writer = new FileWriter("out.txt")) {
11            writer.write("Hello");
12        } catch (IOException e) {
13            e.printStackTrace();
14        }
15        class NotCloseable implements AutoCloseable {
16            void write() {}
17            @Override
18            public void close() {}
19        }
20        try (NotCloseable nc = new NotCloseable()) {
21            nc.write();
22        }
23    }
24 }
```

Input

stdin input

Output

```
FileReader is not allowed - file
I/O is restricted in this
environment.
```

QUESTIONS

Q1
Multiple Catch Blocks Ordering
25 marks**Q2**
Basic Thread Creation and Start
25 marks**Q3**
try with resources Misuse
25 marks**Q4**
Producer-Consumer with
BlockingQueue
25 marks

4/4 answered

Q4 Producer-Consumer with BlockingQueue

25 marks

```
import java.util.concurrent.*;

public class ProducerConsumerDemo {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        Thread producer = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                queue.add(i);
            }
        };

        Thread consumer = new Thread() -> {
            for (int i = 0; i < 10; i++) {
                int val = queue.poll();
                System.out.println("Consumed: " + val);
            }
        };

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("Queue size: " + queue.size());
    }
}
```

Your Code

```
1 import java.util.concurrent.*;
2 public class ProducerConsumerDemo {
3     public static void main(String[] args) throws InterruptedException
4     {
5         BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);
6         Thread producer = new Thread() -> {
7             try {
8                 for (int i = 0; i < 10; i++) {
9                     queue.put(i);
10                }
11            } catch (InterruptedException e) {
12                Thread.currentThread().interrupt();
13            }
14        };
15        Thread consumer = new Thread() -> {
16            try {
17                for (int i = 0; i < 10; i++) {
18                    Integer val = queue.take();
19                    System.out.println("Consumed: " + val);
20                }
21            } catch (InterruptedException e) {
22                Thread.currentThread().interrupt();
23            }
24        };
25        producer.start();
26        consumer.start();
27        producer.join();
28        consumer.join();
29        System.out.println("Queue size: " + queue.size());
30    }
31 }
```

Input

stdin input

Output

```
Consumed: 0
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
```